

SKILL WEEK-1

Session -1

Task1: To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile.

```
ruhi0404@Ruhitha:~/Operating_Systems$ tree
.
├── LICENSE
├── Makefile
├── README.md
├── assets
│   └── architecture.png
├── bin
├── docs
│   ├── design.md
│   └── report.pdf
├── include
│   ├── builtin.h
│   ├── executor.h
│   ├── parser.h
│   └── shell.h
├── obj
├── pra3task1
├── pra3task1.c
├── pra3task2
├── pra3task2.c
├── practical
├── scripts
│   └── run.sh
├── skill
├── src
│   ├── builtin.c
│   ├── executor.c
│   ├── main.c
│   ├── parser.c
│   └── shell.c
├── tests
│   ├── test_executor.c
│   └── test_parser.c
└──
```

11 directories, 22 files
ruhi0404@Ruhitha:~/Operating_Systems\$

Observation:

The Linux development environment was successfully set up and the project directory structure was created. GCC and Git were configured and verified. The project was organized into separate directories for source files, header files, object files, scripts, tests, documentation, and other project resources. The initial MakeFile was also created to support automated compilation and build management. The shell architecture and organization of the Linux project environment were understood.

Task2: To Analyse process abstraction, execute fork(), understand exec() family, analyse parent-child relationships, inspect process tree, and practice system call tracing.

```
ruhi0404@Ruhitha: ~/Operati x + Library Functions Manual exec(3)
NAME
    execl, execlp, execlx, execv, execvp, execvpe - execute a file
LIBRARY
    Standard C library (libc, -lc)
SYNOPSIS
    #include <unistd.h>

    extern char **environ;

    int execl(const char *pathname, const char *arg, ...
        /*, (char *) NULL */);
    int execlp(const char *file, const char *arg, ...
        /*, (char *) NULL */);
    int execlx(const char *pathname, const char *arg, ...
        /*, (char *) NULL, char *const envp[] */);
    int execv(const char *pathname, char *const argv[]);
    int execvp(const char *file, char *const argv[]);
    int execvpe(const char *file, char *const argv[], char *const envp[]);

    Feature Test Macro Requirements for glibc (see feature_test_macros(7)):

    execvpe():
        _GNU_SOURCE

DESCRIPTION
    The exec() family of functions replaces the current process image with a new process image. The functions described in this manual page are layered on top of execve(2). (See the manual page for execve(2) for further details about the replacement of the current process image.)

    The initial argument for these functions is the name of a file that is to be executed.

    The functions can be grouped based on the letters following the "exec" prefix.

    l - execl(), execlp(), execlx()
    The const char *arg and subsequent ellipses can be thought of as arg0, arg1, ..., argn. Together they describe a list of one or more pointers to null-terminated strings that represent the argument list available to the executed program. The first argument, by convention, should point to the filename associated with the file being executed. The list of arguments must be terminated by a null pointer, and, since these are variadic functions, this pointer must be cast (char *) NULL.
    Manual page exec(3) line 1 (press h for help or q to quit)
```

```
ruhi0404@Ruhitha: ~/Operati x + _GNU_SOURCE
DESCRIPTION
    The exec() family of functions replaces the current process image with a new process image. The functions described in this manual page are layered on top of execve(2). (See the manual page for execve(2) for further details about the replacement of the current process image.)

    The initial argument for these functions is the name of a file that is to be executed.

    The functions can be grouped based on the letters following the "exec" prefix.

    l - execl(), execlp(), execlx()
    The const char *arg and subsequent ellipses can be thought of as arg0, arg1, ..., argn. Together they describe a list of one or more pointers to null-terminated strings that represent the argument list available to the executed program. The first argument, by convention, should point to the filename associated with the file being executed. The list of arguments must be terminated by a null pointer, and, since these are variadic functions, this pointer must be cast (char *) NULL.

    By contrast with the 'l' functions, the 'v' functions (below) specify the command-line arguments of the executed program as a vector.

    v - execv(), execvp(), execvpe()
    The char *const argv[] argument is an array of pointers to null-terminated strings that represent the argument list available to the new program. The first argument, by convention, should point to the filename associated with the file being executed. The array of pointers must be terminated by a null pointer.

    e - execlx(), execvpe()
    The environment of the new process image is specified via the argument envp. The envp argument is an array of pointers to null-terminated strings and must be terminated by a null pointer.

    All other exec() functions (which do not include 'e' in the suffix) take the environment for the new process image from the external variable environ in the calling process.

    p - execlp(), execvp(), execvpe()
    These functions duplicate the actions of the shell in searching for an executable file if the specified filename does not contain a slash (/) character. The file is sought in the colon-separated list of directory pathnames specified in the PATH environment variable. If this variable isn't defined, the path list defaults to a list that includes the directories returned by confstr(_CS_PATH) (which typically returns the value "/bin:/usr/bin") and possibly also the current working directory; see NOTES for further details.

    execvpe() searches for the program using the value of PATH from the caller's environment, not from the envp argument.

    If the specified filename includes a slash character, then PATH is ignored, and the file at the specified pathname is executed.
    Manual page exec(3) line 27 (press h for help or q to quit)
```

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    printf("==== PROCESS ABSTRACTION AND SYSTEM CALLS =====\n");
    printf("Parent Process Started\n");
    printf("Parent PID : %d\n", getpid());
    printf("Parent PPID : %d\n\n", getppid());

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        // Child process
        printf("----- CHILD PROCESS ----- \n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());

        printf("\nChild is executing 'ls -l' using exec().\n\n");

        execlp("ls", "ls", "-l", NULL);

        // This runs only if exec() fails
        perror("exec failed");
        exit(1);
    }
    else {
        // Parent process
        printf("----- PARENT PROCESS ----- \n");
    }
}

```

🔍 Help
📄 Write Out
📁 Where Is
✂️ Cut
🔍 Execute
📍 Location
↶ Undo

ruhi04040Ruhitha:~/Operating_Systems\$./skillw3task2

==== PROCESS ABSTRACTION AND SYSTEM CALLS =====

Parent Process Started

Parent PID : 2008

Parent PPID : 398

----- PARENT PROCESS -----

Parent PID : 2008

Child PID : 2009

Parent waiting for child to complete...

----- CHILD PROCESS -----

Child PID : 2009

Parent PID : 2008

Child is executing 'ls -l' using exec().

total 100

```

-rw-r--r-- 1 ruhi0404 ruhi0404    0 Aug 10 12:52 LICENSE
-rw-r--r-- 1 ruhi0404 ruhi0404    0 Aug 10 12:52 Makefile
-rw-r--r-- 1 ruhi0404 ruhi0404    0 Aug 10 12:52 README.md
drwxr-xr-x 2 ruhi0404 ruhi0404 4096 Aug 10 12:52 assets
drwxr-xr-x 2 ruhi0404 ruhi0404 4096 Aug 10 12:51 bin
drwxr-xr-x 2 ruhi0404 ruhi0404 4096 Aug 10 12:52 docs
drwxr-xr-x 2 ruhi0404 ruhi0404 4096 Aug 10 12:52 include
drwxr-xr-x 2 ruhi0404 ruhi0404 4096 Aug 10 12:51 obj
-rwxr-xr-x 1 ruhi0404 ruhi0404 16264 Aug  8 03:17 pra3task1
-rw-r--r-- 1 ruhi0404 ruhi0404   930 Aug  8 03:16 pra3task1.c
-rwxr-xr-x 1 ruhi0404 ruhi0404 16136 Aug  8 03:18 pra3task2
-rw-r--r-- 1 ruhi0404 ruhi0404   377 Aug  8 03:18 pra3task2.c
drwxr-xr-x 2 ruhi0404 ruhi0404 4096 Aug 10 12:43 practical
drwxr-xr-x 2 ruhi0404 ruhi0404 4096 Aug 10 12:52 scripts
drwxr-xr-x 2 ruhi0404 ruhi0404 4096 Aug 10 12:43 skill
-rwxr-xr-x 1 ruhi0404 ruhi0404 16304 Aug 10 13:01 skillw3task2
-rw-r--r-- 1 ruhi0404 ruhi0404 1177 Aug 10 13:01 skillw3task2.c
drwxr-xr-x 2 ruhi0404 ruhi0404 4096 Aug 10 12:52 src
drwxr-xr-x 2 ruhi0404 ruhi0404 4096 Aug 10 12:52 tests

```

Child process completed.

Parent process completed.

ruhi04040Ruhitha:~/Operating_Systems\$

Observation: The process abstraction was successfully analysed using fork(), creating a parent-child process relationship. The exec() family was used to execute a new program in the child process, while wait() allowed the parent process to wait for the child to complete. The process hierarchy was inspected using process-tree commands, and system calls were successfully traced using strace.

Session -2

Task1: To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, and Test Interactive Loop.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    char input[100];

    printf("==== SIMPLE INTERACTIVE SHELL ==== \n");

    while (1) {
        // Display prompt
        printf("myshell> ");
        fflush(stdout);

        // Read user input
        if (fgets(input, sizeof(input), stdin) == NULL) {
            printf("\nInput error or EOF.\n");
            break;
        }

        // Remove newline
        input[strcspn(input, "\n")] = '\0';

        // Handle empty input
        if (strlen(input) == 0) {
            continue;
        }

        // Handle exit condition
        if (strcmp(input, "exit") == 0) {
            printf("Exiting shell...\n");
            break;
        }

        // Display entered command
        printf("You entered: %s\n", input);
    }
}
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute

```
    }

    // Display entered command
    printf("You entered: %s\n", input);
}

printf("Shell terminated successfully.\n");

return 0;
}
|
```

^G Help ^O Write Out ^W Where Is ^K Cut

```
Parent process completed.
ruhi0404@Ruhitha:~/Operating_Systems$ man exec
ruhi0404@Ruhitha:~/Operating_Systems$ nano skillw3task2.c
ruhi0404@Ruhitha:~/Operating_Systems$ man exec
ruhi0404@Ruhitha:~/Operating_Systems$ cd ~/Operating_Systems
ruhi0404@Ruhitha:~/Operating_Systems$ nano skill2w1task1.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc skill2w1task1.c -o skill2w1task1
ruhi0404@Ruhitha:~/Operating_Systems$ ./skill2w1task1
===== SIMPLE INTERACTIVE SHELL =====
myshell> hello
You entered: hello
myshell> ruhitha
You entered: ruhitha
myshell> ls
You entered: ls
myshell> OSSp
You entered: OSSp
myshell> exit
Exiting shell...
Shell terminated successfully.
ruhi0404@Ruhitha:~/Operating_Systems$ |
```

Observation: The interactive shell loop was successfully created and tested. The program continuously displayed a prompt, accepted user input, and processed the entered commands. The exit condition was implemented to terminate the loop, demonstrating the basic control flow of an interactive shell.

Task 2: Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction

```

#include <stdio.h>
#include <unistd.h>
#include <termios.h>
#include <string.h>

#define BUFFER_SIZE 100

int main() {
    char buffer[BUFFER_SIZE];
    int index = 0;
    char ch;

    struct termios old_terminal, new_terminal;

    // Save current terminal settings
    tcgetattr(STDIN_FILENO, &old_terminal);

    // Disable canonical mode and echo
    new_terminal = old_terminal;
    new_terminal.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &new_terminal);

    printf("==== KEYBOARD INPUT HANDLER ====\n");
    printf("Type a command and press Enter.\n");
    printf("Press Backspace to delete a character.\n");
    printf("Type 'exit' to terminate.\n\n");

    while (1) {
        index = 0;
        memset(buffer, 0, BUFFER_SIZE);

        printf("myshell> ");
        fflush(stdout);

        while (1) {
            ch = getchar();

```

```

        }

        printf("Command entered: %s\n", buffer);
        break;
    }

    // Handle Backspace
    else if (ch == 127 || ch == '\b') {
        if (index > 0) {
            index--;
            buffer[index] = '\0';

            // Erase character from terminal
            printf("\b \b");
            fflush(stdout);
        }
    }

    // Store normal characters
    else if (index < BUFFER_SIZE - 1) {
        buffer[index++] = ch;
        putchar(ch);
        fflush(stdout);
    }
}

exit_shell:

// Restore terminal settings
tcsetattr(STDIN_FILENO, TCSANOW, &old_terminal);

printf("Shell terminated successfully.\n");

return 0;
}

```

⌘G Help
⌘O Write Out
⌘W Where Is
⌘K Cut
⌘T Execute
⌘G Location
⌘U Undo

```

ruhi0404@Ruhitha:~/Operating_Systems$ nano skill2w1task1.c
ruhi0404@Ruhitha:~/Operating_Systems$ nano skill2w1task2.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc skill2w1task2.c -o skill2w1task2
ruhi0404@Ruhitha:~/Operating_Systems$ ./skill2w1task2
===== KEYBOARD INPUT HANDLER =====
Type a command and press Enter.
Press Backspace to delete a character.
Type 'exit' to terminate.

myshell> hellp
Command entered: hellp
myshell> hello world
Command entered: hello world
myshell> hello
Command entered: hello
myshell> hellllo
Command entered: hellllo
myshell> exit
Exiting shell...
Shell terminated successfully.
ruhi0404@Ruhitha:~/Operating_Systems$ |

```

Observation: The keyboard input handler was successfully implemented using character-by-character input. The program maintained an input buffer, processed the Enter key to complete commands, and handled Backspace to remove previously entered characters. Multi-character commands were successfully accepted and displayed, and the exit command was used to terminate the interactive shell.

Task 3: To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction.

```

ruhi0404@Ruhitha:~/Operating_Systems$ nano skill1task2.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc skill1task2.c -o skill1task2
ruhi0404@Ruhitha:~/Operating_Systems$ ./skill1task2
Interactive Input
Type a command (type 'exit' to quit):
> helllo
You entered: helllo
> welcome
You entered: welcome
> exit
You entered: exit

Program terminated.
ruhi0404@Ruhitha:~/Operating_Systems$ |

```

```
GNU nano 7.2 skill1task2.c
#include <stdio.h>
#include <unistd.h>
#include <termios.h>

#define BUFFER_SIZE 100

int main()
{
    char buffer[BUFFER_SIZE];
    int index = 0;
    char ch;

    struct termios oldt, newt;

    // Get current terminal settings
    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;

    // Disable canonical mode and echo
    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    printf("Interactive Input\n");
    printf("Type a command (type 'exit' to quit):\n");
    printf("> ");

    while (1)
    {
        // Capture keyboard input
        read(STDIN_FILENO, &ch, 1);

        // Handle Enter key
        if (ch == '\n' || ch == '\r')
        {
            buffer[index] = '\0';
            printf("\nYou entered: %s\n", buffer);
        }
    }
}
```

GNU nano 7.2 skill1task2.c

Read 85 lines

Help Exit Write Out Read File Where Is Replace Cut Paste Execute Justify Location Go To Line

```
GNU nano 7.2 skill1task2.c
// Capture keyboard input
read(STDIN_FILENO, &ch, 1);

// Handle Enter key
if (ch == '\n' || ch == '\r')
{
    buffer[index] = '\0';

    printf("\nYou entered: %s\n", buffer);

    // Check exit command
    if (index == 4 &&
        buffer[0] == 'e' &&
        buffer[1] == 'x' &&
        buffer[2] == 'i' &&
        buffer[3] == 't')
    {
        break;
    }

    // Reset input buffer
    index = 0;

    printf("> ");
}

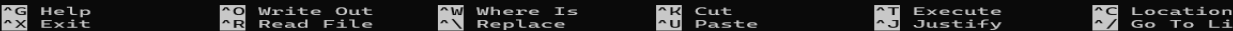
// Handle Backspace
else if (ch == 127 || ch == 8)
{
    if (index > 0)
    {
        index--;

        // Erase character from terminal
        printf("\b \b");
        fflush(stdout);
    }
}
```

GNU nano 7.2 skill1task2.c

Help Exit Write Out Read File Where Is Replace Cut Paste Execute Justify Location Go To Line M-U Un M-E Re

```
        index = 0;
        printf("> ");
    }
    // Handle Backspace
    else if (ch == 127 || ch == 8)
    {
        if (index > 0)
        {
            index--;
            // Erase character from terminal
            printf("\b \b");
            fflush(stdout);
        }
    }
    // Handle normal characters
    else if (index < BUFFER_SIZE - 1)
    {
        buffer[index++] = ch;
        // Display typed character
        printf("%c", ch);
        fflush(stdout);
    }
}
// Restore original terminal settings
tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
printf("\nProgram terminated.\n");
return 0;
}
```



Observation

The program successfully captures keyboard input character by character and stores it in an input buffer. It handles the **Backspace** key by removing the previously entered character and processes the **Enter** key to execute the entered command. The program supports **multi-character commands** and displays the entered input correctly. The exit command terminates the interactive program successfully.